

Ejercicio 7

Investigación REACT

¿Qué es Jest y React Testing Library?

Jest es un framework de pruebas de JavaScript, creado por Meta, que permite escribir y ejecutar pruebas automáticas. Trae todo incluido: ejecutor de tests, funciones de verificación (*assertions*) y herramientas para simular (*mock*) funciones o módulos.

React Testing Library (RTL) es una librería que permite probar componentes de React desde el punto de vista del usuario: en lugar de revisar el funcionamiento interno del componente (su estado o sus métodos), verifica lo que se ve en pantalla y cómo se interactúa con él (textos, botones, inputs).

Se usan juntas: RTL renderiza el componente y busca elementos en la pantalla, y Jest ejecuta la prueba y verifica que el resultado sea el esperado. Sirven para detectar errores tempranamente, evitar que cambios nuevos rompan lo que ya funcionaba y dar confianza al modificar el código.

¿Cuáles son sus características y para qué sirve?

Características de Jest:

- ☆ Configuración mínima: viene listo para usar en proyectos como Create React App (en Vite se instala aparte).
- ☆ Sintaxis simple con **describe**, **test** (o **it**) y **expect**.
- ☆ Incluye *matchers* como **toBe**, **toEqual** y **toBeTruthy**.
- ☆ Permite crear *mocks* de funciones y módulos (**jest.fn()**).
- ☆ Muestra reportes claros y puede medir la cobertura de código (*coverage*).
- ☆ Ejecuta las pruebas en paralelo y tiene modo *watch*, que vuelve a correrlas al guardar cambios.

Características de React Testing Library:

- ☆ Prueba el comportamiento visible del componente y no su implementación interna.
- ☆ Busca elementos como lo haría un usuario: **getByText**, **getByRole**, **getByLabelText**.
- ☆ Simula acciones del usuario con **fireEvent** o **userEvent** (clics, escritura).
- ☆ Fomenta buenas prácticas de accesibilidad, al buscar elementos por su rol y etiqueta.
- ☆ Con **@testing-library/jest-dom** agrega verificaciones como **toBeInTheDocument()**.

Ejemplo de implementación:

Componente a probar (**Contador.jsx**):

```
1  import { useState } from "react";
2
3  function Contador() {
4    const [contador, setContador] = useState(0);
5
6    return (
7      <div>
8        <p>Valor actual: {contador}</p>
9        <button onClick={() => setContador(contador + 1)}>Sumar</button>
10     </div>
11   );
12 }
13
14 export default Contador;
15
```

Prueba (**Contador.test.jsx**):

```
1  import { render, screen, fireEvent } from "@testing-library/react";
2  import "@testing-library/jest-dom";
3  import Contador from "../Contador";
4
5  describe("Componente Contador", () => {
6    test("muestra el valor inicial en 0", () => {
7      render(<Contador />);
8      expect(screen.getByText("Valor actual: 0")).toBeInTheDocument();
9    });
10
11    test("incrementa el valor al hacer clic en el botón", () => {
12      render(<Contador />);
13      fireEvent.click(screen.getByText("Sumar"));
14      expect(screen.getByText("Valor actual: 1")).toBeInTheDocument();
15    });
16  });
17
```

Para ejecutarlas:

npm test

Explicación: **describe** agrupa las pruebas del componente **Contador** y cada **test** define un caso. En el primero, **render** monta el componente en un entorno simulado y **screen.getByText** busca el texto en pantalla; **expect(...).toBeInTheDocument()** verifica que el valor inicial sea 0. En el segundo, **fireEvent.click** simula que el usuario presiona el botón "Sumar" y se comprueba que el texto pase a "Valor actual: 1". Si el componente dejara de funcionar como se espera, la prueba fallaría y avisaría del error.